

APB Protocol Slave Module (SRAM) Verification

APB Protocol Slave Module (SRAM) Verification

Interface

Transaction

Driver

Monitor

Generator

Agent

Agent Package

Environment Package

Environment

Scoreboard

Test-lib Package

Base Test

Fixed Test

Read and Write from the same Address Test

Reset Test

Mid Operation Reset Test

Boundary Test

Overwrite Test

Tb Top

Interface

```
`timescale 1ns/1ps

interface apb_if #(parameter DATA_BUS_WIDTH=32, ADDR_BUS_WIDTH=32)
    (
        input logic pclk
    );

    logic presetn;
    logic psel;
    logic penable;
    logic pwrite;

    logic [DATA_BUS_WIDTH-1:0] pwdata;
    logic [ADDR_BUS_WIDTH-1:0] paddr;
    logic [DATA_BUS_WIDTH-1:0] prdata;

    logic pready;
    logic pslverr;

    task apply_reset(int cycles = 2);
        presetn = 0;
        repeat(cycles) @(posedge pclk);
        presetn = 1;
    endtask

endinterface
```

Transaction

```
`timescale 1ns/1ps

class transaction #(parameter DATA_BUS_WIDTH=32, ADDR_BUS_WIDTH=32);
    rand bit pwrite;
    rand logic [ADDR_BUS_WIDTH-1:0] paddr;
    rand logic [DATA_BUS_WIDTH-1:0] pwdata;
    bit psel;
endclass
```

```

bit penable;

logic [DATA_BUS_WIDTH-1:0] prdata;
    bit pready;
    bit pslverr;

    bit [DATA_BUS_WIDTH-1:0] exp_prdata;
    bit exp_pready;
    bit exp_pslverr;

function void display(string tag);
    $display("[%0t] [%0s] psel=%0b penable=%0b pwrite=%0b paddr=%0h
pwdata=%0h | prdata=%0h pready=%0b pslverr=%0b", $time, tag,
    psel, penable, pwrite, paddr, pwdata,
    prdata, pready, pslverr
    );

endfunction
endclass

```

Driver

```

`timescale 1ns/1ps
class driver #(
    parameter DATA_BUS_WIDTH=32, ADDR_BUS_WIDTH=32
);
    mailbox gen2drv;
    virtual apb_if #(DATA_BUS_WIDTH, ADDR_BUS_WIDTH) vif;
    transaction #(DATA_BUS_WIDTH, ADDR_BUS_WIDTH) tr;

    function new(mailbox gen2drv, virtual apb_if #(DATA_BUS_WIDTH,
ADDR_BUS_WIDTH) vif);
        this.gen2drv = gen2drv;
        this.vif      = vif;
        $display("[%0t] DRIVER CONSTRUCTED", $time);
    endfunction

    task run;
    forever begin
        gen2drv.get(tr);
    end
endclass

```

```

// SETUP phase:
@(posedge vif.pclk);
vif.psel    <= 1'b1;
vif.penable <= 1'b0;
vif.pwrite  <= tr.pwrite;
vif.paddr   <= tr.paddr;
vif.pwdata  <= tr.pwdata;

// ACCESS phase
@(posedge vif.pclk);
vif.penable <= 1'b1;

@(posedge vif.pclk);
wait (vif.pready);
@(negedge vif.pclk);

tr.prdata = vif.prdata;
tr.pready = vif.pready;
tr.pslverr = vif.pslverr;

tr.display("DRIVER");

@(posedge vif.pclk);
vif.psel    <= 1'b0;
vif.penable <= 1'b0;
end
endtask
endclass

```

Monitor

```

`timescale 1ns/1ps
class monitor #(parameter DATA_BUS_WIDTH=32, ADDR_BUS_WIDTH=32
);
    mailbox mon2sb;
    virtual apb_if #(DATA_BUS_WIDTH, ADDR_BUS_WIDTH) vif;

    function new(mailbox mon2sb, virtual apb_if #(DATA_BUS_WIDTH,
ADDR_BUS_WIDTH) vif);
        this.mon2sb = mon2sb;

```

```

        this.vif      = vif;
        $display("[%0t] MONITOR CONSTRUCTED", $time);
    endfunction
task run();

    forever begin

        transaction #(DATA_BUS_WIDTH, ADDR_BUS_WIDTH) mon_tr;
        mon_tr = new();

        // Wait for SETUP phase
        do @(posedge vif.pclk);
        while (!(vif.psel && !vif.penable));

        @(negedge vif.pclk);
        mon_tr.psel  = vif.psel;
        mon_tr.pwrite = vif.pwrite;
        mon_tr.paddr = vif.paddr;
        mon_tr.pwdata = vif.pwdata;

        // Wait for ACCESS phase
        @(posedge vif.pclk);
        @(posedge vif.pclk);
        wait(vif.psel && vif.penable);

        mon_tr.penable = vif.penable;

        // Wait for transfer completion
        wait(vif.pready);

        mon_tr.prdata  = vif.prdata;
        mon_tr.pready  = vif.pready;
        mon_tr.pslverr = vif.pslverr;

        mon_tr.display("MONITOR");

        mon2sb.put(mon_tr);

    end

endtask

endclass

```

Generator

```
class generator #(
    parameter DATA_BUS_WIDTH = 32,
    parameter ADDR_BUS_WIDTH = 32
);

    mailbox gen2drv;
    int count;
    virtual apb_if #(DATA_BUS_WIDTH, ADDR_BUS_WIDTH) vif;

    function new(mailbox gen2drv, virtual apb_if #(DATA_BUS_WIDTH,
ADDR_BUS_WIDTH) vif);
        this.gen2drv = gen2drv;
        this.vif=vif;
    endfunction

task run(
    input bit use_test_addr = 0,
    input bit [ADDR_BUS_WIDTH-1:0] test_addr[] = {},
        input bit test_pwrite[] = {}
);

    for (int i = 0; i < count; i++) begin
        transaction #(DATA_BUS_WIDTH, ADDR_BUS_WIDTH) tr;
        tr = new();
        if (!tr.randomize() with {
            paddr inside {[0:128]};
        })
            $error("RANDOMIZATION FAILED");

            if (use_test_addr) begin
                tr.paddr = test_addr[i];
                tr.pwrite = test_pwrite[i];
            end
        tr.display("GENERATOR");
        gen2drv.put(tr);
    end
endtask

task write();
```

```

repeat(count) begin
    transaction #(DATA_BUS_WIDTH, ADDR_BUS_WIDTH) tr;
    tr=new();

    if(!tr.randomize() with {
        paddr inside {[0:63]};
        pwrite==1;
    })
        $error("RANDOMIZATION FAILED");

    tr.display("GENERATOR");
    gen2drv.put(tr);
end

endtask

task read();

repeat(count) begin
    transaction #(DATA_BUS_WIDTH, ADDR_BUS_WIDTH) tr;
    tr=new();

    if(!tr.randomize() with {
        paddr inside {[0:63]};
        pwrite==0;
    })
        $error("RANDOMIZATION FAILED");

    tr.display("GENERATOR");
    gen2drv.put(tr);
end

endtask
endclass

```

Agent

```

`timescale 1ns/1ps
class agent #(parameter DATA_BUS_WIDTH=32, ADDR_BUS_WIDTH=32);

    driver drv;

```

```

        monitor mon;
        generator gen;

        mailbox gen2drv;
        virtual apb_if #(DATA_BUS_WIDTH, ADDR_BUS_WIDTH) vif;

        function new(mailbox mon2sb, virtual apb_if #(DATA_BUS_WIDTH,
ADDR_BUS_WIDTH) vif);
            this.vif=vif;
            gen2drv=new();
            drv=new(gen2drv, vif);
            mon=new(mon2sb, vif);
            gen=new(gen2drv, vif );
        endfunction

        task run();
            fork
                gen.run();
                drv.run();
                mon.run();
            join_none
        endtask

    endclass

```

Agent Package

```

package apb_agent_pkg;
    `include "../top/transaction.sv"
    `include "../agent/driver.sv"
    `include "../agent/monitor.sv"
    `include "../seq-lib/generator.sv"
    `include "../agent/agent.sv"
endpackage

```

Environment Package

```

package apb_env_pkg;
    import apb_agent_pkg :: * ;
    `include "../env/scoreboard.sv"
    `include "../env/env.sv"
endpackage

```


Environment

```
`timescale 1ns/1ps

class env #(parameter DATA_BUS_WIDTH=32, ADDR_BUS_WIDTH=32);
    agent agt;
    scoreboard sb;

    mailbox mon2sb;

    function new(virtual apb_if #(DATA_BUS_WIDTH, ADDR_BUS_WIDTH) vif);

        mon2sb=new();
        $display("[%0t] ENV: before agent", $time);
        agt=new(mon2sb, vif);
        $display("[%0t] ENV: after agent", $time);
        $display("[%0t] ENV: before scoreboard", $time);
        sb=new(mon2sb);
        $display("[%0t] ENV: after scoreboard", $time);
    endfunction
endclass
```

Scoreboard

```
class scoreboard #(
    parameter DATA_BUS_WIDTH = 32,
    parameter ADDR_BUS_WIDTH = 32,
    parameter MEMSIZE          = 64,
    parameter MEM_BLOCK_SIZE = 8
);

    mailbox mon2sb;

    bit [MEM_BLOCK_SIZE-1:0] expected_mem [bit [ADDR_BUS_WIDTH-1:0]];

    int compare_count = 0;
    int read_pass = 0;
    int read_fail = 0;
    int error_pass= 0;
```

```

int error_fail= 0;
int ready_pass= 0;
int ready_fail= 0;

localparam int BLOCKS_PER_WORD = DATA_BUS_WIDTH / MEM_BLOCK_SIZE;
localparam int MEM_BYTES      = MEMSIZE * BLOCKS_PER_WORD;

virtual apb_sb_if #(64, 8) sb_vif;
function new(mailbox mon2sb);

    this.mon2sb = mon2sb;

    $display("Scoreboard Constructed");

endfunction

task run();

    forever begin

        transaction #(DATA_BUS_WIDTH, ADDR_BUS_WIDTH) tr;
        bit exp_pready;
        bit exp_pslverr;
        bit [DATA_BUS_WIDTH-1:0] expected_data;

        mon2sb.get(tr);

        // Only process ACCESS phase
        if (tr.psel && tr.penable) begin

            exp_pready  = 1'b1;
            exp_pslverr = 1'b0;
            expected_data = '0;

            //write

            if (tr.pwrite) begin

                if (tr.paddr <= (MEM_BYTES - BLOCKS_PER_WORD)) begin

```

```

        for (int i = 0; i < BLOCKS_PER_WORD; i++) begin

            expected_mem[tr.paddr + i] =
                tr.pwdata[(i * MEM_BLOCK_SIZE) +:
MEM_BLOCK_SIZE];

        end

        exp_pslverr = 1'b0;

        $display(
            "===== DATA WRITE
=====");

        );

    end
    else begin

        exp_pslverr = 1'b1;

    end

end

// READ

else begin

    if (tr.paddr <= (MEM_BYTES - BLOCKS_PER_WORD)) begin

        expected_data = '0;

        for (int i = 0; i < BLOCKS_PER_WORD; i++) begin
            if (expected_mem.exists(tr.paddr + i)) begin
                expected_data[(i * MEM_BLOCK_SIZE) +:
MEM_BLOCK_SIZE] =
                    expected_mem[tr.paddr + i];
            end
            else begin
                expected_data[(i * MEM_BLOCK_SIZE) +:
MEM_BLOCK_SIZE] = '0;

```

```

        end

    end

    exp_pslverr = 1'b0;

    $display(
        "===== DATA READ
=====
");

    end
    else begin

        expected_data = '0;
        exp_pslverr = 1'b1;

    end

    if (tr.prdata == expected_data) begin
        read_pass++;
        $display("[%0t] READ DATA MATCH | ADDR=%0h |
EXPECTED=%0h | ACTUAL=%0h", $time, tr.paddr, expected_data, tr.prdata);
    end
    else begin
        read_fail++;
        $display("[%0t] READ DATA FAIL | ADDR=%0h |
EXPECTED=%0h | ACTUAL=%0h", $time, tr.paddr, expected_data, tr.prdata);
    end
    end

    // Compare PREADY

    if (tr.pready != exp_pready) begin
        ready_fail++;
        $display("[%0t] PREADY FAIL | EXP=%0b ACT=%0b", $time,
exp_pready, tr.pready);
    end
    else
        // $display("[%0t] PREADY CORRECT | EXP=%0b ACT=%0b",
$time, exp_pready, tr.pready);

```

```

        // Compare PSLVERR

        if (tr.pslverr != exp_pslverr) begin
            error_fail++;
            $display("[%0t] PSLVERR FAIL | EXP=%0b ACT=%0b", $time,
exp_pslverr, tr.pslverr);
        end
        compare_count++;
    end

end
endtask

task report();

    $display("");
    $display("=====");
    $display("          SCOREBOARD REPORT");
    $display("=====");
    $display("READ PASS   = %0d", read_pass);
    $display("READ FAIL   = %0d", read_fail);
    $display("PSLVERR FAIL = %0d", error_fail);
    $display("READY FAIL  = %0d", ready_fail);
    $display("=====");

endtask

endclass

```

Test-lib Package

```

package test_lib_pkg;
    `include "../test-lib/base_test.sv"
    `include "../test-lib/read_write.sv"
    `include "../test-lib/deterministic.sv"
    `include "../test-lib/reset_test.sv"
    `include "../test-lib/mid_reset.sv"
    `include "../test-lib/overwrite.sv"
    `include "../test-lib/boundary_test.sv"
endpackage

```

Base Test

```

import apb_env_pkg::*;

class base_test #(
    parameter DATA_BUS_WIDTH = 32,
    parameter ADDR_BUS_WIDTH = 32
);

    env env_o;
    int total_transactions;
    virtual apb_if #(DATA_BUS_WIDTH, ADDR_BUS_WIDTH) vif;

    function new(virtual apb_if #(DATA_BUS_WIDTH, ADDR_BUS_WIDTH) vif);
        this.vif=vif;
        $display("[%0t] INSIDE THE BASE TEST CONSTRUCTOR", $time);
        env_o = new(vif);
    endfunction

    virtual task run();
        env_o.agt.gen.count = total_transactions;
        env_o.agt.gen.run();
    endtask

endclass

```

Fixed Test

```

class fixed_test extends base_test #(
    DATA_BUS_WIDTH,
    ADDR_BUS_WIDTH
);

    function new(
        virtual apb_if #(DATA_BUS_WIDTH, ADDR_BUS_WIDTH) vif
    );
        super.new(vif);
    endfunction

    bit [ADDR_BUS_WIDTH-1:0] test_addr[];
    bit test_pwrite[];

    virtual task run();

```

```

        test_addr = new[total_transactions];
        test_addr = '{5, 10, 20, 5, 10, 20, 8, 12, 16, 24, 28, 32, 36, 40, 44,
48, 52, 56, 60, 5}';
        test_pwrite = '{1, 1, 1, 0, 0, 0, 1, 1, 1, 0, 0, 1, 1, 1, 1, 1, 1, 0,
0, 0}';

        $display("[%0t] TOTAL TRANSACTIONS = %0d",
            $time, total_transactions);

        // RUN TEST
        env_o.agt.gen.count = total_transactions;
        env_o.agt.gen.run(1, test_addr, test_pwrite);

    endtask

endclass

```

Read and Write from the same Address Test

```

class read_write extends base_test #(
    DATA_BUS_WIDTH,
    ADDR_BUS_WIDTH
);

    function new(
        virtual apb_if #(DATA_BUS_WIDTH, ADDR_BUS_WIDTH) vif
    );
        super.new(vif);
    endfunction

    bit [ADDR_BUS_WIDTH-1:0] test_addr[];
    bit test_pwrite[];

    virtual task run();

        test_addr = new[total_transactions];
        for (int i=0; i<total_transactions/2; i++) begin
            test_addr[i]=i; //half writei
            test_pwrite[i] = 1;
        end
    endtask

```

```

    test_addr[i + total_transactions/2] = i; //half read
    test_pwrite[i + total_transactions/2] = 0;
end

    $display("[%0t] TOTAL TRANSACTIONS = %0d",
            $time, total_transactions);

    // RUN TEST
    env_o.agt.gen.count = total_transactions;
    env_o.agt.gen.run(1, test_addr);

endtask

endclass

```

Reset Test

```

class reset_test extends base_test #(
    DATA_BUS_WIDTH,
    ADDR_BUS_WIDTH
);

    function new(
        virtual apb_if #(DATA_BUS_WIDTH, ADDR_BUS_WIDTH) vif
    );
        super.new(vif);
    endfunction

    bit [ADDR_BUS_WIDTH-1:0] test_addr[];
    bit test_pwrite[];

task run();

    env_o.agt.gen.count = 0;

    vif.presetn = 0;
    $display("RESET ASSERTED. PREADY=%0b", vif.pready);

    repeat(2) @(posedge vif.pclk);

    // Check reset behavior
    if (vif.pready != 0)
        $display("RESET TEST FAIL: PREADY");

```



```

    if (vif.pslverr != 0)
        $display("RESET TEST FAIL: PSLVERR");

    else $display("RESET TEST PASSED. PREADY= %0b, PSLVERR= %0b",
vif.pready, vif.pslverr);

    vif.presetn = 1;

    repeat(2) @(posedge vif.pclk);
    $display("RESET DEASSERTED. PREADY=%0b", vif.pready);

    test_addr    = new[2];
    test_pwrite = new[2];

    test_addr    = '{5, 5};
    test_pwrite = '{1, 0};

    env_o.agt.gen.count = 5;
    env_o.agt.gen.run(1, test_addr, test_pwrite);
    $display("END OF RESET TEST AT [%0t]", $time);
    $finish;

endtask
endclass

```

Mid Operation Reset Test

```

class mid_reset_test extends base_test #(
    DATA_BUS_WIDTH,
    ADDR_BUS_WIDTH
);
    function new(
        virtual apb_if #(DATA_BUS_WIDTH, ADDR_BUS_WIDTH) vif
    );
        super.new(vif);
    endfunction
    bit [ADDR_BUS_WIDTH-1:0] test_addr[];
    bit test_pwrite[];
    virtual task run();
        test_addr    = new[1];
        test_pwrite = new[1];
    endtask
endclass

```

```

test_addr    = '{4}';
test_pwrite  = '{1}';
env_o.agt.gen.count = 1;
env_o.agt.gen.run(1, test_addr, test_pwrite);

@(posedge vif.pclk);

// Assert reset in the middle
vif.presetn = 0;
$display("[%0t] RESET ASSERTED", $time);
repeat(2) @(posedge vif.pclk);

// Check DUT
if (vif.pready != 0)
    $display("MID RESET TEST FAIL: PREADY");
if (vif.pslverr != 0)
    $display("MID RESET TEST FAIL: PSLVERR");
else
    $display("MID RESET TEST PASSED. PREADY=%0b, PSLVERR=%0b",
vif.pready, vif.pslverr);

vif.presetn = 1;
repeat(2) @(posedge vif.pclk);
$display("[%0t] RESET DEASSERTED", $time);

test_addr    = '{4}';
test_pwrite  = '{0}';
env_o.agt.gen.count = 1;
env_o.agt.gen.run(1, test_addr, test_pwrite);

$display("[%0t] END OF MID RESET TEST", $time);
$finish;
endtask
endclass

```

Boundary Test

```

class boundary_test extends base_test #(
    DATA_BUS_WIDTH,
    ADDR_BUS_WIDTH
);
    function new(
        virtual apb_if #(DATA_BUS_WIDTH, ADDR_BUS_WIDTH) vif

```

```

);
    super.new(vif);
endfunction
bit [ADDR_BUS_WIDTH-1:0] test_addr[];
bit test_pwrite[];
virtual task run();
    test_addr = new[4];
    test_pwrite = new[4];
    test_addr = '{252, 252, 253, 253};
    test_pwrite = '{1, 0, 1, 0};

    env_o.agt.gen.count = 4;
    env_o.agt.gen.run(1, test_addr, test_pwrite);
    repeat(4 * 60) @(posedge vif.pclk);

    $display("[%0t] END OF BOUNDARY ADDR TEST", $time);
    $finish;
endtask
endclass

```

Overwrite Test

```

class overwrite_test extends base_test #(
    DATA_BUS_WIDTH,
    ADDR_BUS_WIDTH
);
    function new(
        virtual apb_if #(DATA_BUS_WIDTH, ADDR_BUS_WIDTH) vif
    );
        super.new(vif);
    endfunction
    bit [ADDR_BUS_WIDTH-1:0] test_addr[];
    bit test_pwrite[];
    virtual task run();
        // Write same address 3 times with different data, then read once
        test_addr = new[4];
        test_pwrite = new[4];
        test_addr = '{4, 4, 4, 4};
        test_pwrite = '{1, 1, 1, 0};

        env_o.agt.gen.count = 4;
    endtask
endclass

```

```

        env_o.agt.gen.run(1, test_addr, test_pwrite);
        repeat(4 * 60) @(posedge vif.pclk);
        $display("[%0t] END OF SAME ADDR OVERWRITE TEST", $time);
        $finish;
    endtask
endclass

```

Tb Top

```

`timescale 1ns/1ps

module tb_top #(parameter DATA_BUS_WIDTH=32,
parameter ADDR_BUS_WIDTH=32,
parameter TOTAL_TRANSACTIONS = 30
);
    import test_lib_pkg::*;

    bit pclk=0;
    bit presetn;
    base_test test;
    read_write rw;
    fixed_test ft;
    reset_test rt;
    mid_reset_test wt;
    overwrite_test ot;
    boundary_test bt;

    always #5 pclk=~pclk;

    apb_if #(.DATA_BUS_WIDTH(DATA_BUS_WIDTH),
.ADDR_BUS_WIDTH(ADDR_BUS_WIDTH)) vif(pclk);

    //DUT

    apb_v3_sram #(
        .ADDR_BUS_WIDTH(ADDR_BUS_WIDTH) ,
        .DATA_BUS_WIDTH(DATA_BUS_WIDTH),
        .MEMSIZE (64),
        .MEM_BLOCK_SIZE (8),
        .RESET_VAL (0),

```

```

        .EN_WAIT_DELAY_FUNC    (1),
        .MIN_RAND_WAIT_CYC     (2),
        .MAX_RAND_WAIT_CYC     (2)
    ) DUT (.PCLK(vif.pclk), .PRESETn(vif.presetn),
          .PREADY(vif.pready), .PSLVERR(vif.pslverr),
        .PRDATA(vif.prdata),
          .PSEL(vif.psel), .PENABLE(vif.penable),
        .PWRITE(vif.pwrite),
          .PADDR(vif.paddr), .PWDATA(vif.pwdata)
    );

//DETERMINE WHICH TEST TO RUN

task run_test();

if ($test$plusargs("RW")) begin
    $display("[%0t] RUNNING READ_WRITE TEST", $time);
    rw = new(vif);
    test = rw;

end
else if ($test$plusargs("FIXED")) begin
    $display("[%0t] RUNNING FIXED TEST", $time);
    ft=new(vif);
    test=ft;

end
else if ($test$plusargs("RESET")) begin
    $display("[%0t] RUNNING RESET TEST", $time);
    rt=new(vif);
    test=rt;

end
else if ($test$plusargs("MID")) begin
    $display("[%0t] RUNNING MID TRANSACTION RESET CHECK TEST",
$time);
    wt=new(vif);
    test=wt;

end
else if ($test$plusargs("OW")) begin
    $display("[%0t] RUNNING OVERWRITE TEST", $time);
    ot=new(vif);
    test=ot;

end

```

```

else if ($test$plusargs("BOUNDARY")) begin
    $display("[%0t] RUNNING BOUNDARY TEST", $time);
    bt=new(vif);
    test=bt;
end

else begin
    $display("[%0t] RUNNING BASE TEST", $time);
    test = new(vif);
end
endtask

initial begin
    $dumpfile("waveform.vcd");
    $dumpvars(1, tb_top);
end

initial begin

    vif.apply_reset();
    run_test();
    test.total_transactions = TOTAL_TRANSACTIONS;

    fork
        test.env_o.agt.mon.run();
        test.env_o.agt.drv.run();
        test.env_o.sb.run();
    join_none

    test.run();

    wait(test.env_o.sb.compare_count == test.total_transactions);
    test.env_o.sb.report();
    $display("[%0t] ENV: WAIT CONDITION SATISFIED", $time);

    $finish;

end

endmodule

```